

Croppico

Code Document

We have 2 process running on croppico device

1. A angular UI build which is displayed all the time in the device with homescreen, settings, and maintenance
2. A python script to handle all UI requests from the angular which works as the backend for the angular and also send data/alarm payload to cloud via MQTT.

Both angular and py script runs on same device and communicate locally

The py script is running as a service and the angular is opened on boot up in a browser

```
pi@raspberrypi:~$ sudo su
root@raspberrypi:/home/pi# cd /etc/systemd/system
root@raspberrypi:/etc/systemd/system# ls
autologin@.service      dbus-org.freedesktop.timesync1.service  graphical.target.wants  poweroff.target.wants  sockets.target.wants  systemd-resolved.service.wants
bluetooth.target.wants  default.target                          halt.target.wants       printer.target.wants   socketxtp_install.sh  timers.target.wants
croppico-api.service    default.target.wants                    multi-user.target.wants  processdata.sqlite     socketxtp.service
dbus-fi.wl.wpa_supplicant1.service  display-manager.service  network-online.target.wants  rc-local.service.d     sshd.service
dbus-org.bluez.service  getty.target.wants                     ota.service              reboot.target.wants    sysinit.target.wants
dbus-org.freedesktop.Avahi.service  getty@tty1.service.d      pitunnel.service          remote-fs.target.wants  syslog.service
root@raspberrypi:/etc/systemd/system# cat croppico-api.service
[Unit]
Description=Croppico-API
After=multi-user.target

[Service]
Type=simple
ExecStart=/usr/bin/python /home/pi/croppico-api-new/main_dsrc_uart_ui.py
Restart=on-failure
RestartSec=2s

[Install]
WantedBy=multi-user.target
root@raspberrypi:/etc/systemd/system#
```

The angular prod build UI should be placed in the /var/www/html directory

1. Py script code flow:

Three major things will be done by the backend Py script

- A. Interaction with angular UI which is running locally
- B. A loop constantly running to get the data from microcontroller connected via UART to receive DATA and ALARM payload and send it to the cloud via MQTT
- C. Will receive command from cloud/app via MQTT and process the request like turning on pump etc.

Major file structure:

1. [main.py](#)
2. [slave_controller.py](#)
3. Slave_request.py
4. Connection_handler.py

Code FLOW:

- A. UI -> py -> controller:

Any interaction with the UI will be handled by the [main.py](#) script via flask as API's. The basic functionality of the UI is to just display the live values and settings of the controller and even we can change the setting through UI.

UI -> icon/button press -> flask api called([main.py](#)) -> command to microcontroller is sent via UART if needed -> return value to UI.

B. Py <-> cloud:

SLAVE_CONTROLLER.PY will poll data from the controller via UART every 15 seconds where we will receive DATA and ALARM data from the controller.

The received data and alarm will be sent to the cloud via MQTT.

This SLAVE_CONTROLLER.PY will use a support py file named SLAVE_REQUEST.py where the UART payload format for communication between py and microcontroller is defined.

CODE EXPLANATION:

Modules Overview

1. `main.py`

****Purpose:**** Flask-based API server exposing endpoints to interact with the system.

This is the entry point for HTTP-based interaction with the controller. It creates a Flask application and serves as the bridge between UI and backend logic.

Key Components:

- - GET /data: Fetches real-time sensor and status data from the `SlaveController`.
- - POST /light`: Controls individual or grouped lights.
- - GET /getsettings: Retrieves current configuration/settings from the slave.
- - GET /historicalData/<start>/<end>: Queries historical sensor data from local SQLite DB.
- - POST /system/screen: Pauses or resumes polling of the slave.
- - POST /system/restart: Sends a command to restart the slave controller.
- - POST /system/flush and `POST /system/topup`: Commands to trigger or stop tank water.
- - POST /system/connectwifi: Connect the system to Wi-Fi using SSID and password from the client through the UI.

Key Variables:

- - device_id: Fetched via `constants.getserial()`—used to identify the device in MQTT topics and logs.
- - slave: An instance of `SlaveController` which performs hardware interaction and command execution to the controller(stm32).

2. `slave\_controller.py`

****Purpose:**** Central threaded controller class responsible for continuous polling and interaction with the slave MCU. It manages data logging, light schedules, sensor reads, actuator control, and MQTT publishing.

Class: `SlaveController(Thread)`

Inherits Python's `Thread` class, meaning its `run()` method is executed on a separate thread.

Constructor Details:

- - Initializes UART on /dev/ttyUSB0 at 115200 baud.
- - Connects to both local and cloud MQTT brokers.
- - Initializes logger, SQLiteDict storage, and flags like `__run_controller`, `__isUserControlling`.

Core Loop (``run()``):

- - Runs indefinitely if `__run_controller` is True.
- - Performs scheduled light control based on time-of-day.
- - Reads sensor data every 10 seconds from the controller via UART.
- - Publishes data to cloud if new information is available every 15-20minutes.
- - Logs new readings to ``sensor_data.db`` periodically.
- - Publish the DATA and ALARM payload received from the controller to cloud via MQTT.

Major Functions:

- - `pollSlave(request, need_response=True)`: Communicates with the microcontroller.
- - `checkSlaveStatus()`: Gets sensor readings and alarms.
- - `insert_data(...)`: Stores PH, EC, temp data in SQLite.
- - `setLightStatus(light, state)`: Controls lights by sending command to controller via UART.
- - `setNewSettings(...)`: Handles new thresholds for pH, EC, water, etc.
- - `cloudMessage(...)`: MQTT downlink handler, routes config and adhoc control messages.
- - `processAdhoc(...)`: Handles adhoc JSON commands for triggering peripheral control like pumps or top-up.

3. ``connection_handler.py``

****Purpose:**** Handles MQTT-based communication with the cloud (via `mqtt.Client()`).

Class: ``ConnectionHandler``

- - Connects to cloud broker on construction.
- - Listens for messages under topic ``server/<device_id>/#``.
- - Publishes updates to topic: ``croppico/<device_id>/<dtype>``

Key Functions:

- - `connect()`: Starts MQTT loop and subscribes to appropriate topics.
- - `publish(dtype, data)`: Publishes device data (sensor, alarm, settings, etc.).
- - `on_connect(...)`: Callback for MQTT connection success.
- - `on_message(...)`: Callback that passes incoming MQTT payload.

4. ``local_publish.py``

****Purpose:**** Similar to `connection_handler.py`, but used for publishing to a local MQTT broker (for debug/dev).

Class: ``Publish``

- - Connects to local broker at `mqtt.denvik.in`.
- - Username/password hardcoded (``Denvik``, password).
- - Exposes `publish(topic, data)` function to send debug info like pH and EC debug logs.

5. ``slave_requests.py``

****Purpose:**** Acts as a command library for UART interaction with the slave MCU.

Classes:

- - Requests: Container for command categories ('Maintenance', 'Lights', 'Settings') and decoders.
- - Maintenance: Methods to format commands for water/chiller/supplement pumps.
- - Lights: Functions for controlling brightness and switching.
- - Settings: For setting EC, pH, water thresholds, supplement doses, and calibration inputs.
- - Decode: Converts CSV strings from slave MCU into Python dictionaries.

Example Commands:

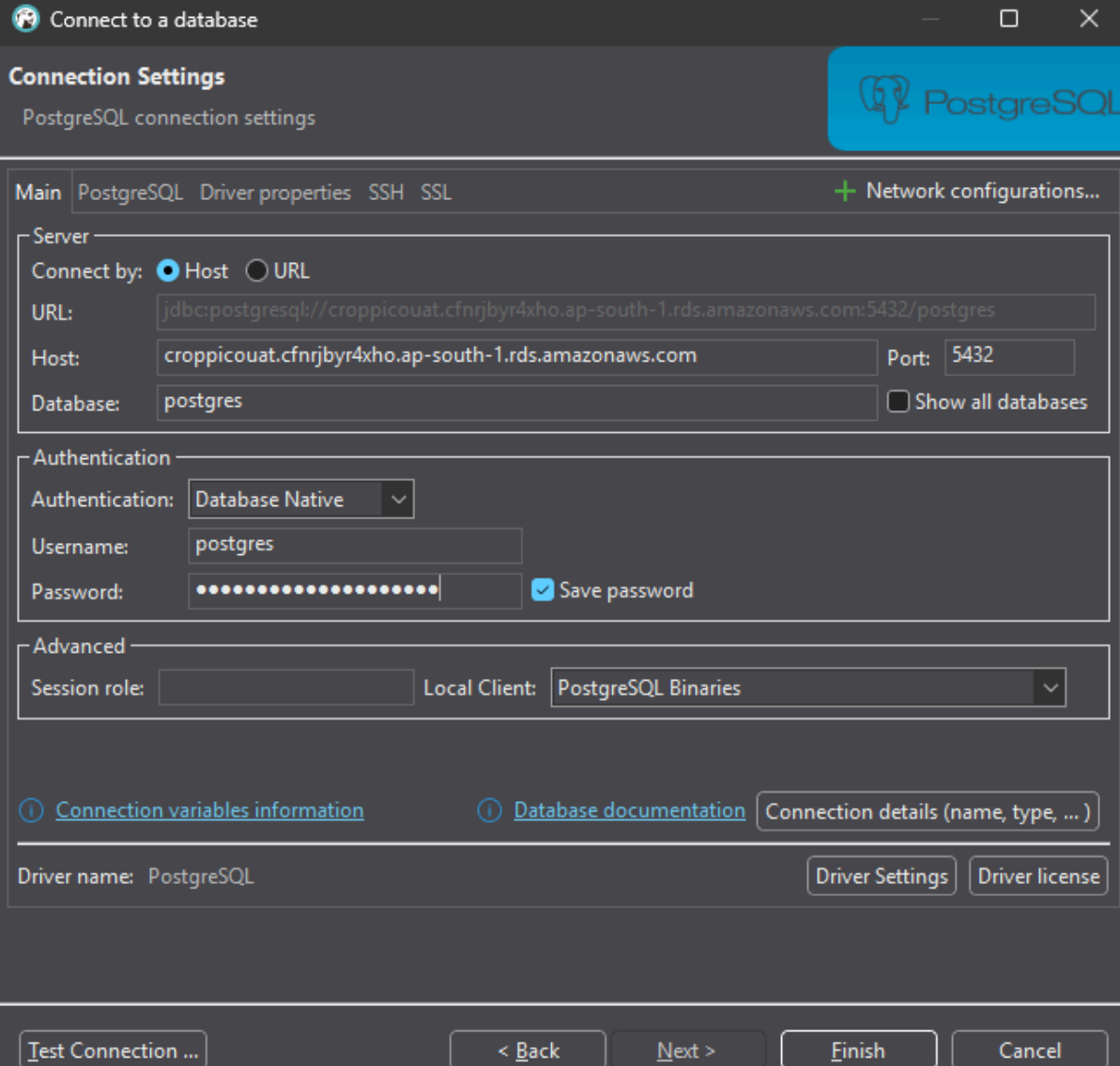
- - `"00,11"`: Get sensor data.
- - `"00,02,03,<ecMin>,<ecIdeal>,<error>,<pumpError>"`: Set EC threshold.
- - `"00,02,06,01,<phVal>"`: Calibrate pH with known value.

Steps to do in production for each device

1. [py](#) script - running as service
2. Angular - prod build placed in /var/www/html directory
3. cloud registration - Register in postgres db (connect to db using dbeaver with croppico creds and run 2 sql query)
4. Remote registration - Socketxp registration (using raja account and exe file)

Cloud device registration:

1. Connect to croppico db using dbeaver



The screenshot shows the 'Connect to a database' dialog box in DBeaver. The title bar says 'Connect to a database'. The main heading is 'Connection Settings' with a subtitle 'PostgreSQL connection settings'. There is a PostgreSQL logo in the top right corner. The dialog has several tabs: 'Main', 'PostgreSQL', 'Driver properties', 'SSH', and 'SSL'. The 'Main' tab is selected. Below the tabs, there is a '+ Network configurations...' button. The 'Server' section has 'Connect by:' with 'Host' selected (radio button) and 'URL' (radio button). The 'URL' field contains 'jdbc:postgresql://croppicouat.cfnrjbyr4xho.ap-south-1.rds.amazonaws.com:5432/postgres'. The 'Host' field contains 'croppicouat.cfnrjbyr4xho.ap-south-1.rds.amazonaws.com' and the 'Port' field contains '5432'. The 'Database' field contains 'postgres' and there is a checkbox for 'Show all databases'. The 'Authentication' section has 'Authentication:' set to 'Database Native' (dropdown), 'Username:' set to 'postgres', and 'Password:' field with masked characters. There is a checkbox for 'Save password' which is checked. The 'Advanced' section has 'Session role:' (empty field) and 'Local Client:' set to 'PostgreSQL Binaries' (dropdown). At the bottom, there are links for 'Connection variables information', 'Database documentation', and 'Connection details (name, type, ...)'. Below these links, 'Driver name:' is set to 'PostgreSQL', and there are buttons for 'Driver Settings' and 'Driver license'. At the very bottom, there are buttons for 'Test Connection ...', '< Back', 'Next >', 'Finish', and 'Cancel'.

Connect to a database

Connection Settings
PostgreSQL connection settings

Main PostgreSQL Driver properties SSH SSL + Network configurations...

Server

Connect by: ☒ Host ☐ URL

URL: jdbc:postgresql://croppicouat.cfnrjbyr4xho.ap-south-1.rds.amazonaws.com:5432/postgres

Host: croppicouat.cfnrjbyr4xho.ap-south-1.rds.amazonaws.com Port: 5432

Database: postgres ☐ Show all databases

Authentication

Authentication: Database Native

Username: postgres

Password: ☒ Save password

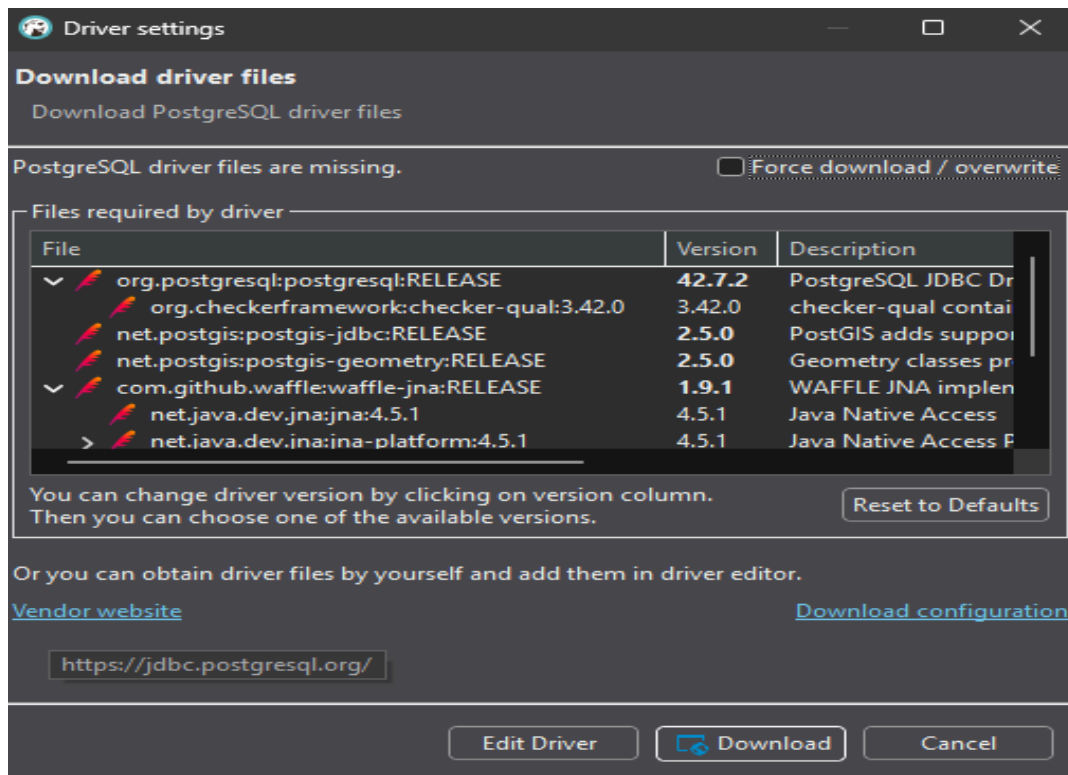
Advanced

Session role: Local Client: PostgreSQL Binaries

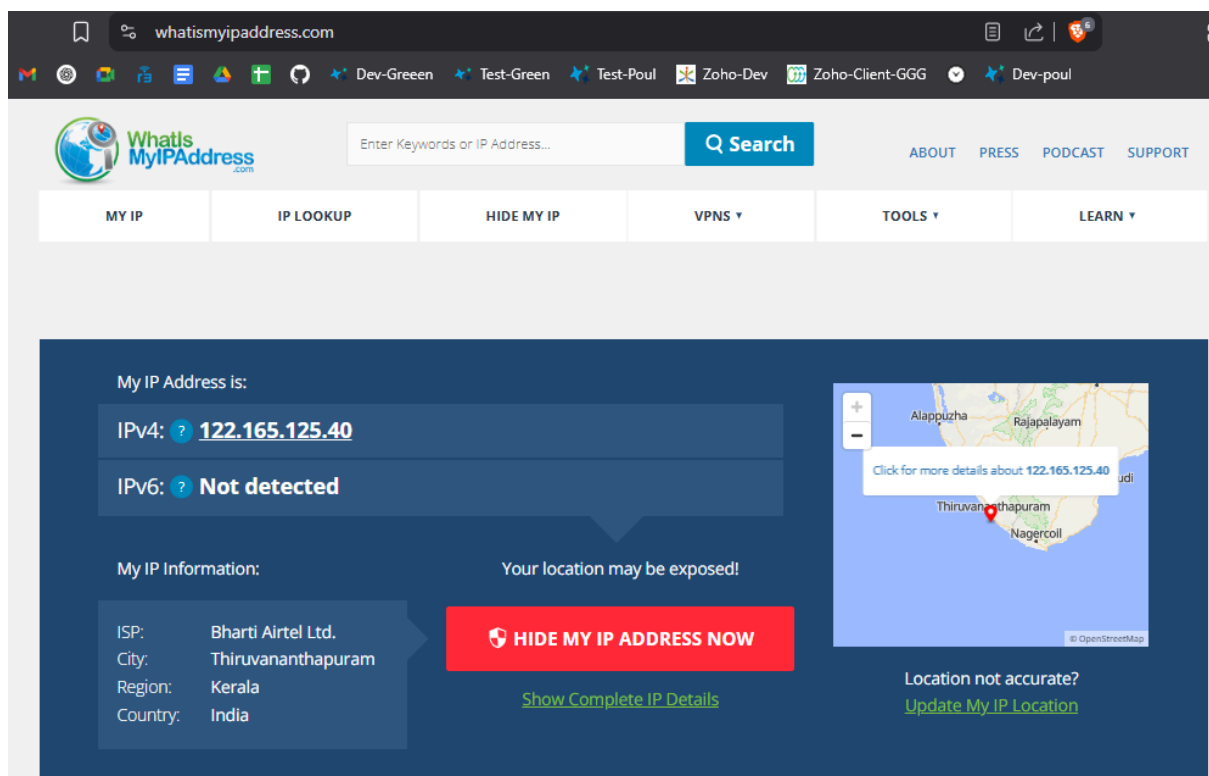
[Connection variables information](#) [Database documentation](#) Connection details (name, type, ...)

Driver name: PostgreSQL Driver Settings Driver license

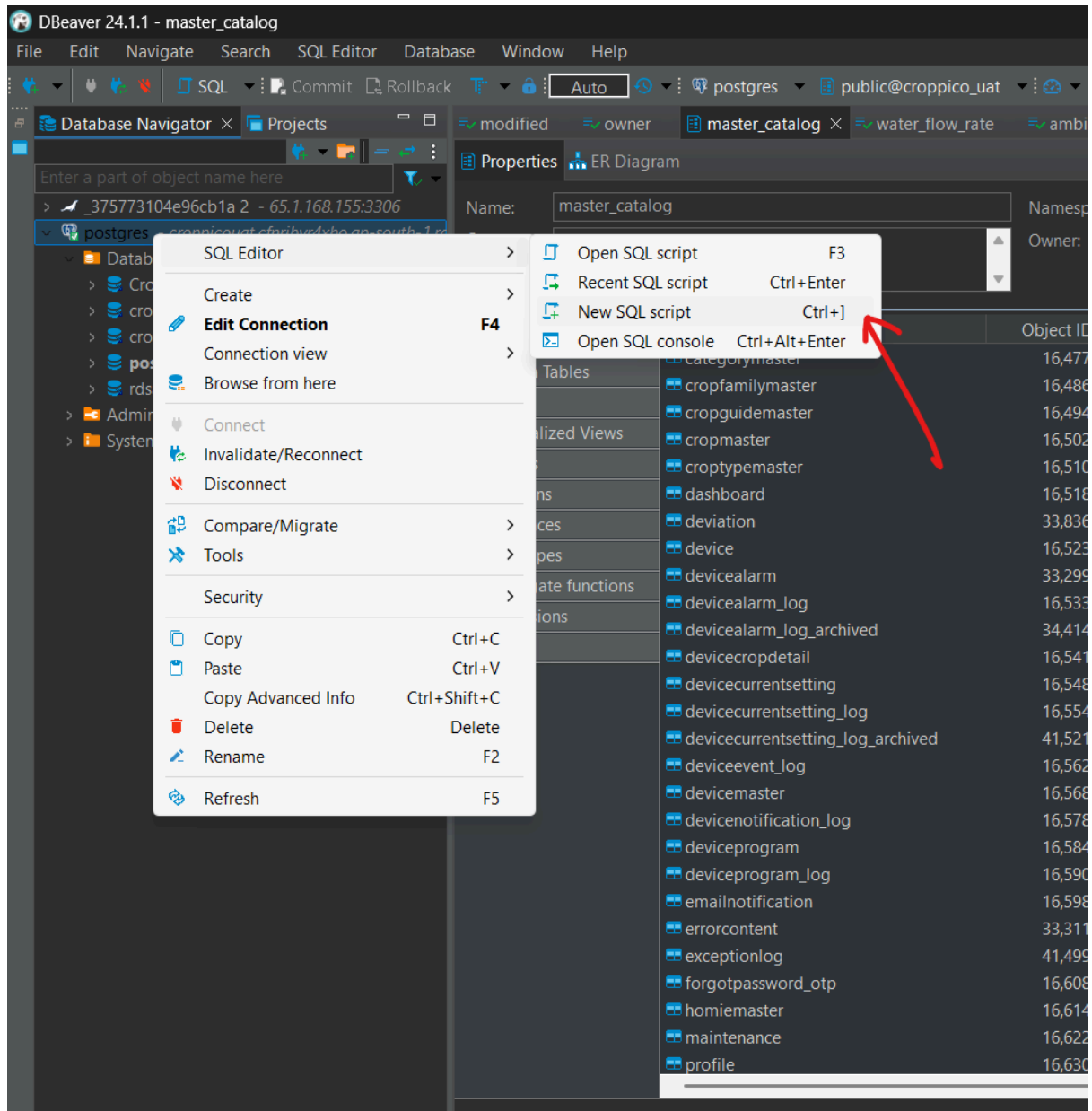
Test Connection ... < Back Next > Finish Cancel

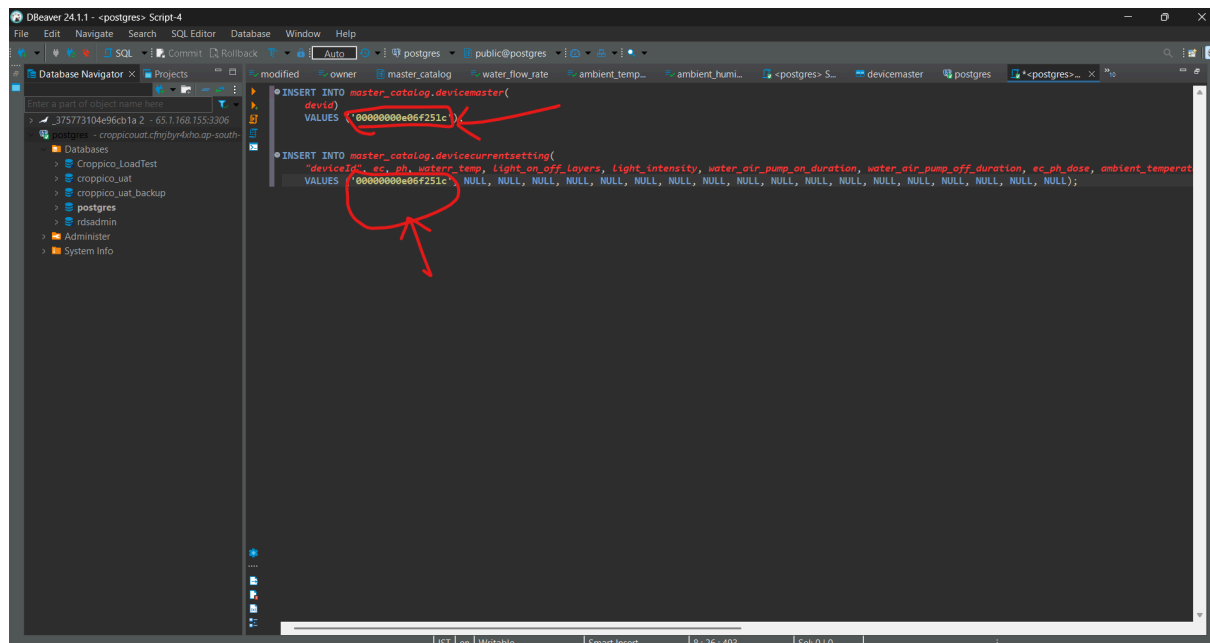


- We can only access the db when pc/laptop is connected to Denvik wifi and public ip should be 122.165.125.40



3. Run two queries to register new device in db





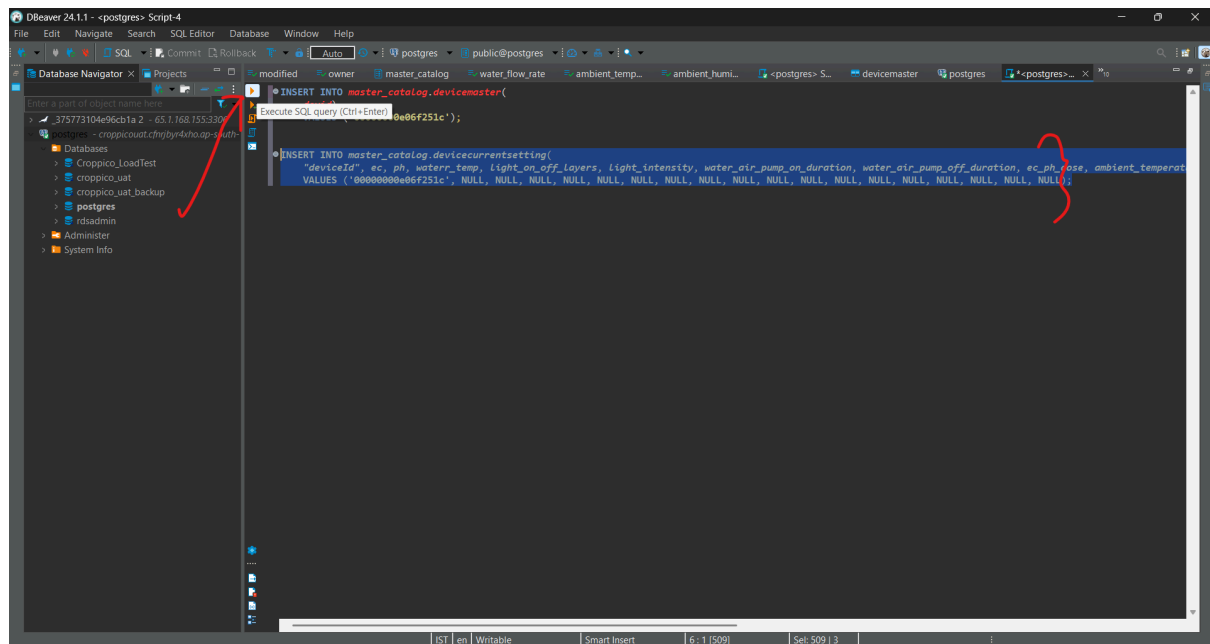
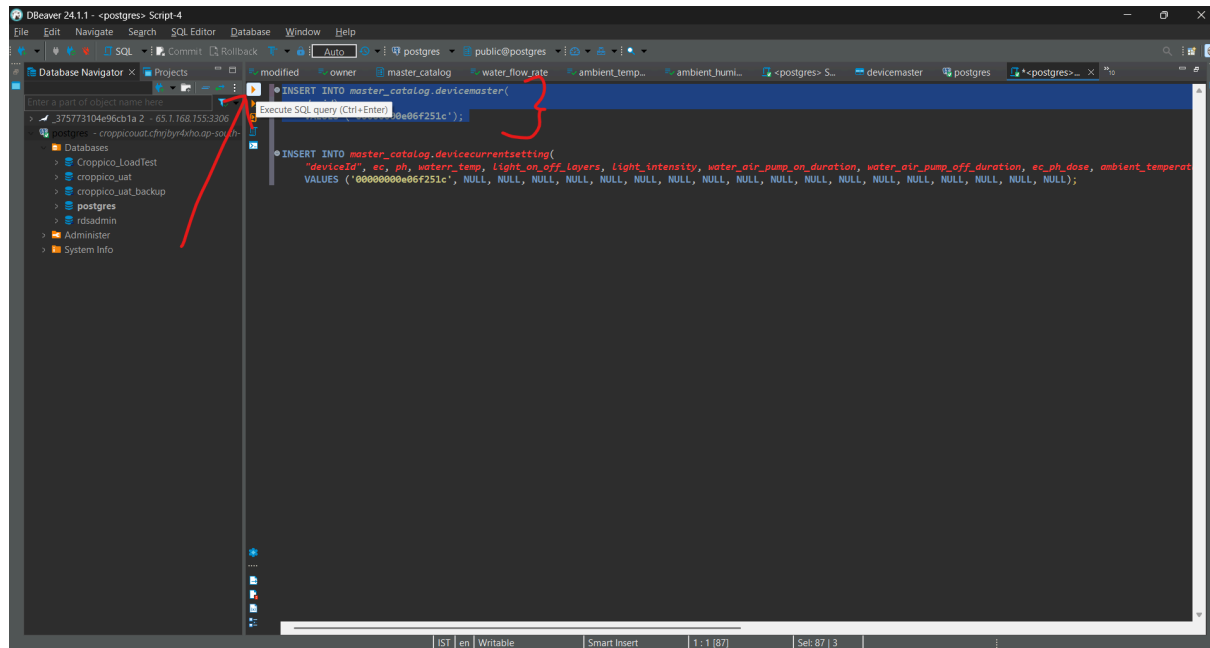
```
INSERT INTO master_catalog.devicemaster(  
    deviceid  
VALUES ('00000000e06f251c');
```



```
INSERT INTO master_catalog.devicecurrentsetting(  
    "deviceid", ec, ph, waterr_temp, light_on_off_layers, light_intensity,  
water_air_pump_on_duration, water_air_pump_off_duration, ec_ph_dose,  
ambient_temperature, ambient_humidity, water_flow_rate, supp_ph_inc_lev,  
supp_ph_dec_lev, supp_ec_a_lev, supp_ec_b_lev, "hasAlarm", created_timestamp,  
water_level_indicator)  
VALUES ('00000000e06f251c', NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL,  
NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL);
```

Replace device id with new device id

Run 2 queries individually and cross check whether it is reflected in db



Socketxp installation and maintenance documentation:

Operator Docs

- ☒ Croppico Maintatence Application - User Manual
- ☒ Installation of SocketXp - Croppico

Developer Docs

- ☒ SocketXP Connection
- ☒ Installation of SocketXp_DEV